

Claude 연동 가이드

코딩을 몰라도 이 프로젝트를 계속 개발하는 방법
설치 · 프로젝트 열기 · 실전 지시문 6개 · 주의사항

작성일	2026-08-26
대상	방준현 (클라이언트)
필요	Node.js + Claude 구독 또는 API 크레딧
함께 볼 것	『Conatus 인수인계서』 PDF

장 번호(E-1 ~ E-5)는 저장소의 `HANDOVER.md` 와 같습니다. Claude 에게 “*HANDOVER.md E-4 의 프롬프트 ④대로 해줘*” 처럼 그대로 지시할 수 있습니다.

이 문서는 무엇인가

이 프로젝트(conatusipsi.com)는 **Claude Code** 라는 도구로 개발됐습니다. 터미널에 **한국어로 지시하면 Claude 가 코드를 직접 읽고 고치고 테스트하고 배포**합니다. 코딩을 몰라도 쓸 수 있게, 설치부터 실제로 복사해 쓸 수 있는 지시문까지 순서대로 적었습니다.

프로젝트의 현재 상태·계정·운영 방법은 **별도 PDF 『Conatus 인수인계서』**에 있습니다. 두 문서 모두 저장소 루트의 `HANDOVER.md` 한 파일에서 만들어집니다.

미리 알아 둘 것 3가지

1. Claude Code 사용료와, 서비스 안의 AI 기능(학생에게 보이는 AI 분석) 사용료는 **별개**입니다.
2. `main` 에 코드를 올리면 **곧바로 실서비스에 반영**됩니다. 확인 없이 올리게 두지 마세요.
3. 무엇이든 모르겠으면 Claude에게 한국어로 그대로 물어보면 됩니다. *"이거 위험한 작업이야?"*, *"되돌릴 수 있어?"*가 가장 유용한 질문입니다.

E. Claude로 개발 이어가기

이 프로젝트는 **Claude Code** 로 개발됐습니다. 같은 방식으로 이어가면 Claude가 코드를 읽고 직접 수정·테스트·배포까지 해 줍니다. 코딩을 몰라도 **한국어로 지시**하면 됩니다.

E-1. 설치 (한 번만)

1. **Node.js 설치** — nodejs.org 에서 LTS 버전 다운로드 후 설치
2. **Claude Code 설치** — claude.com/code 안내에 따라 설치 (터미널에서 `npm install -g @anthropic-ai/claude-code`)
3. **Claude 계정 로그인** — 터미널에서 `claude` 실행 → 안내에 따라 로그인 (Claude 구독 또는 Anthropic API 크레딧 필요)

E-2. 프로젝트 열기

터미널(맥: 터미널 / 윈도우: PowerShell)에서 아래를 순서대로 입력합니다.

```
# 1) 코드 내려받기 (최초 1회)
git clone https://github.com/conatusipsi-byte/prismedu-kr-admission.git
cd prismedu-kr-admission

# 2) 필요한 프로그램 설치 (최초 1회, 몇 분 걸림)
npm install

# 3) 환경변수 파일 만들기 (최초 1회)
#   Vercel 대시보드의 환경변수 값을 복사해 .env.local 파일에 붙여넣기.
#   형식은 .env.local.example 파일 참고. 또는 아래 명령으로 자동 내려받기:
npx vercel env pull .env.local --scope joonhyeon-s-projects

# 4) Claude Code 실행
claude
```

이후에는 `cd prismedu-kr-admission` → `claude` 두 줄이면 됩니다.

로컬에서 사이트를 직접 띄워보려면 `npm run dev` → 브라우저에서 `http://localhost:9002` . (로컬 개발 서버도 **실제 원격 Supabase DB에 연결**되니, 데이터 수정 작업은 주의하세요.)

E-3. Claude가 자동으로 참고하는 규칙 파일

저장소 최상단의 `CLAUDE.md` 에 이 프로젝트의 규칙이 정리돼 있습니다. Claude Code는 **매번 이 파일을 자동으로 읽고** 작업합니다. 예를 들어 이런 것들이 들어 있습니다:

- 기술 스택과 폴더 구조

- 브랜드 색상·디자인 톤
- **정직성 원칙** — 데이터가 없으면 "모른다"고 말하고, 수치를 지어내지 않는다
- **ETL 주의사항** — 데이터 적재 시 정시 데이터가 지워지지 않게 하는 방법
- 진학사·대학어디가 크롤링 금지 등 법적 주의사항

규칙을 바꾸고 싶으면 Claude에게 "CLAUDE.md에 ○○ 규칙을 추가해줘"라고 하면 됩니다.

E-4. 실전 예시 프롬프트 6개

그대로 복사해서 Claude Code에 붙여넣으면 됩니다.

① 프로젝트 파악하기 (제일 먼저 해볼 것)

이 프로젝트 구조를 설명해줘. HANDOVER.md와 CLAUDE.md를 먼저 읽고,
어떤 기능이 어느 폴더에 있는지, 지금 안 되는 기능이 뭔지 정리해서 알려줘.

② 기능 추가하기

학과 검색(/admissions)에 "모집인원 순으로 정렬" 기능을 추가해줘.
기존 필터 UI 패턴과 일관되게 만들고, 작업 끝나면 npm run typecheck와 npm test를 돌려서
통과하는지 확인해줘.

팁: 기능을 말할 때 **화면 경로**(/admissions) 나 **화면에 보이는 글자**를 같이 알려주면 Claude가 정확한 파일을 훨씬 빨리
찾습니다.

③ 합격률 기능 상태 확인·개선

합격률 기능이 지금 어떤 데이터를 근거로 계산되는지 코드로 확인해서 설명해줘.
"합격자 표본 기반"으로 전환하려면 무엇이 더 필요한지, 학생 제보 입력 UI를 만든다면
어느 파일을 건드려야 하는지 알려줘. HANDOVER.md B장 1번 항목을 참고해.

④ 새 대학 모집요강 PDF 적재하기 (ETL)

새 대학 모집요강 PDF를 적재하려고 해. 파일 경로는 ○○○ 이고 대학은 ○○대야.

scripts/etl/ 의 기존 절차(텍스트 추출 → 구조화 JSON → dept-matcher 매칭 → load-admissions)를 그대로 따라줘.

반드시 지킬 것:

- load-admissions.ts 실행 시 --only=<대학ID> 를 꼭 붙일 것 (정시 데이터 덮어쓰기 방지)
- --execute 없이 먼저 dry-run 으로 결과를 보여주고, 내 확인을 받은 뒤에 실행할 것
- 학과 매칭이 애매하면 추정하지 말고 건너뛰고 리포트할 것
- 적재 후 정시 617행이 그대로 남아있는지 검증할 것

⑤ 12월 정시 확정본으로 교체하기

정시 데이터를 12월 확정본(정시모집요강)으로 교체하려고 해.

지금 DB에는 예정안(planStatus="preliminary")이 16개 대학 617행 들어있어.

scripts/etl/load-jeongsi-plan.ts 를 읽고, 확정본 교체 절차를 먼저 설명해줘.

- 기존 수시 데이터(93교 4,173행)가 절대 지워지지 않는 방식(JSONB 병합)인지 확인
- planStatus 를 preliminary → finalized 로 바꾸는 지점
- --commit 없이 dry-run 먼저

설명을 듣고 내가 확인한 다음에 실행할게.

⑥ 다음 학년도(2028)로 갱신하기 — 매년 필요

2028학년도 데이터로 갱신하려고 해. HANDOVER.md 의 C장(학년도 갱신)을 먼저 읽고

아래 3단계를 순서대로 진행해줘.

- 1) 2028 모집요강 PDF 를 ETL 로 적재 (load-admissions.ts 는 --only 필수, dry-run 먼저)
- 2) lib/admission/current-year.ts 의 LATEST_LOADED_ADMISSION_YEAR 를 2028 로 상향
- 3) Vercel 환경변수 NEXT_PUBLIC_ADMISSION_YEAR 를 2028 로 변경 (3개 타깃 모두) 후 재배포

각 단계마다 내 확인을 받고 진행하고, 끝나면 학과 상세 API 의 "year" 가 2028 로 바뀌었는지, 검색 결과가 비어있지 않은지 실제로 확인해서 보여줘.

E-5. ⚠ Claude에게 작업시킬 때 꼭 지킬 것

1. DB를 수정하는 작업 전에는 반드시 백업 Supabase 대시보드 → Database → Backups 에서 백업 상태 확인.
Claude에게 "작업 전에 백업부터 하자" 라고 먼저 말하세요. 데이터를 지우거나 통째로 덮어쓰는 작업은 되돌리기 어렵습니다.

2. 배포 전에는 항상 테스트

```
npm run typecheck # 타입 오류 검사
npm test          # 단위 테스트 (2026-08-26 기준 90개 파일 / 980개 테스트 전부 통과)
npm run build     # 빌드 성공 확인
```

숫자는 기능이 늘면 함께 늘어납니다. 중요한 건 "실패 0건" 입니다. 하나라도 빨간색(FAIL)이면 배포하지 마세요. Claude에게 "배포 전에 typecheck·test·build 다 돌려서 통과하는지 보여줘" 라고 하면 알아서 합니다. main 에 푸시하면 바로 실서비스에 반영되므로, 확인 없이 푸시하지 않게 하세요.

3. ETL(데이터 적재) 시 --only 를 반드시 사용 scripts/etl/load-admissions.ts 를 --only 없이 --execute 하면 모든 대학의 전형 데이터를 통째로 다시 씁니다. 이때 나중에 따로 넣은 정시 데이터(617행)가 날아갈 수 있습니다.

```
npx tsx scripts/etl/load-admissions.ts --only=<대학ID> # dry-run (안전, DB 변경 없음)
npx tsx scripts/etl/load-admissions.ts --only=<대학ID> --execute # 실제 적재
```

적재 후에는 정시 행 수가 그대로인지 반드시 확인하세요.

4. AI 기능은 호출할 때마다 돈이 나갑니다 console.anthropic.com → Usage / Billing 에서 잔액을 주기적으로 확인하세요. 잔액이 떨어지면 AI 챗·분석이 mock(가짜) 응답으로 대체됩니다. Claude Code 자체 사용료와 서비스의 AI 기능 사용료는 별개입니다.
5. 모르면 그냥 물어보세요 "방금 한 작업이 무슨 뜻이야?", "이거 위험한 작업이야?", "되돌릴 수 있어?" — Claude는 한국어로 다 설명해 줍니다. 이해가 안 되는 상태로 main 푸시나 DB 수정을 승인하지 마세요.